

Proof Interfaces for Exploratory Mathematics

Nishant Kheterpal Matthew Keenan Cyrus Omar Jean-Baptiste Jeannin
nshkh@umich.edu mckeenan@umich.edu comar@umich.edu jeannin@umich.edu
University of Michigan
Ann Arbor, MI, USA

Abstract

This paper describes a mathematics interface intended for both educational applications and exploratory mathematics. Both learners and mathematics practitioners often use pen and paper or a whiteboard to perform equational reasoning, manipulate expressions, or construct proofs. These workflows lead to a variety of problems: transcription errors, tedious writing and notation, and/or unclear standards for proof justification. We extend the **Hazel Prover**, an equational-reasoning interface in the Hazel live programming environment, to add capabilities for exploratory mathematics, with varying levels of verbosity and automation aimed at both students and expert users. Students require more deliberate practice when learning mathematical concepts and correspondingly more verbose justifications, while experts may benefit from significant mathematical automation. With this in mind, our interface supports multiple levels of mathematical automation and simplification, grounded in a rewrite search architecture. For expert users, motivated by a gap in the accessibility of formal methods, we support proof export to the Rocq theorem prover, extending this rewrite search to proof tactics. This paper closes with several case studies covering elementary- to college-level mathematics.

1 Introduction

Both learners and mathematics practitioners conduct “exploratory math”: the kinds of open-ended rewriting of terms and bounding of quantities that are often done on a whiteboard. This whiteboard work, while frequently useful for creative purposes, suffers from significant friction: rewriting terms is tedious, and small errors (flipping $+$ to $-$, for example) are easily made and hard to notice. Results need to be preserved or shared, by typesetting the derivation in $\text{\LaTeX}$, copying it onto paper for submission, or photographing the whiteboard.

Expert users may conduct math directly in $\text{\LaTeX}$, where copying lines is trivial but the syntax may become tedious, especially for constructs like matrices. Meanwhile, computer algebra systems such as Mathematica [19], SymPy [8], Maple [6], or MATLAB [14] work well to solve expressions or perform fixed, known transformations on expressions or equations, but do not permit the open-ended exploration needed to bound terms or prove theorems.

Students, on the other hand, require more deliberate practice when learning mathematical concepts and correspondingly more verbose justifications and less automation than

experts. These students may still struggle, in particular when learning formal proof, to break down an argument into sufficiently small steps, or to apply new rules correctly [18]. An instructor might structure practice by limiting the available rules or controlling step granularity.

Nonetheless, both groups of users could benefit from a “digital whiteboard” that adds structure to their work. For experts, this interface must permit open-ended exploration while ensuring incremental correctness. Students, on the other hand, require less automation but a different sort of structure: guidance and scaffolding to help practice mathematical concepts while abstracting away others. These needs motivate varying the mathematical capabilities, automation, and simplification the interface provides.

Full-scale proof assistants offer industry-standard verification capabilities but can be difficult to use for non-experts. Rocq [15], Lean [2], and Isabelle [10] have been used in educational settings and to formalize mathematics. However, the learning curve and syntax can challenge even experts. Educational proof assistants with simplified interfaces and abstractions have been built, such as Waterproof, based on Rocq, which has been used to teach real analysis in the past [16, 17]. Similar systems have been built based on Lean [7, 9]. Graphical tools such as Edukera teach proofs visually [13]. Tools aimed beyond proofs such as GeoGebra [4], Graspable Math [12], and MathXpert [1] inspired parts of our design.

In this work, we present an extension of the *Hazel Prover*, an existing equational reasoning interface, to support exploratory mathematics. Past work using the Hazel Prover focused on the development and deployment of the prover in classrooms for teaching structural induction proofs [5]. The Hazel Prover is built in Hazel, a live programming environment that supports direct manipulation of program structure [11]. The existing educational uses of the Hazel Prover led to a natural second question: can a powerful exploratory interface also be used to teach more basic mathematics? Similar levels of automation in the broader Hazel Prover were found to be valuable for teaching in deployments in college-level programming languages courses [5].

For practitioners in applied fields such as aerospace engineering and physics, this work also connects to the accessibility of formal methods. These practitioners regularly engage in rigorous mathematical reasoning — deriving equations of motion, linearizing dynamics, and discretizing continuous systems — yet formal methods training is not part of their background. We aim to lower the barrier to using formal

methods for symbolic reasoning by developing a platform in which users can manipulate equations or expressions and emit a formal proof of their steps at the end.

Our contributions are the design and implementation of an interface for exploratory mathematics; a customizable rewrite search architecture (Section 3) supporting mathematical capabilities organized by educational level and automation ranging from single steps to simplification; formal proof export from step-by-step derivations; and case studies investigating suitability for tasks from order of operations through algebra and calculus to exploratory numerical methods (Section 4). Section 2 describes the interface’s design goals.

2 Design Goals

In order to ensure our interface is useful for both students and experts, it should fit existing workflows (either for learning exercises or exploration) and be straightforward to use. To guide our design, we established a set of design criteria, informed by our past work on the Hazel Prover [5, Section 2]. Our criteria for usability and workflow fit are similar to design goals in that work, but our goals around coverage and proof export are targeted at the needs of numerical methods practitioners and other experts, rather than students.

Our design goals are: First, *low setup cost*: a new user should be able to begin solving representative tasks with little configuration or theorem-prover-specific training. Second, *low interaction overhead*: routine manipulations such as rewrites, differentiation, and substitution should not require too many actions. Third, *step granularity*: users should be able to take steps that are neither too large (simplifying a full expression) nor too small (only commuting terms). Fourth, *mathematical coverage*: the interface should support the symbolic tasks in our case studies (Section 4).

3 Architecture

Once an expression has been entered, users begin each step by selecting an expression or subexpression and opening a dropdown menu. In this dropdown, experts might seek a useful rewrite or simplification, or students might be asked to apply specific rules, such as distribution or trigonometric identities. The interface supports applying or proposing transformations, checking their permissibility, and recording accepted derivations for proof export. Upon applying or proposing a valid rewrite, the user may take another step.

For example, consider the expression $(2x - 3)(x + 4)$. In a student-facing exercise, Hazel might present distribution as the next available rule. An instructor might, at first, permit students learning binomial expansion to apply only one simple distribution, producing $(2x - 3) \cdot x + (2x - 3) \cdot 4$. But a later lesson might have students do more: $2x^2 - 3x + 8x - 12$, or perhaps a version with like terms collected: $2x^2 + 5x - 12$. An expert user would find this tedious and might prefer simply to click a button that says “Simplify”. These uses differ in

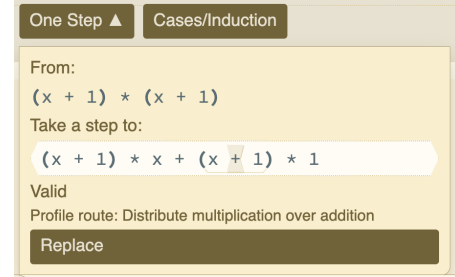


Figure 1. One Step permits only individual rewrites.

how much work Hazel performs, but both require the same basic guarantee: the accepted result must be correct, and an accepted derivation may be exported to Rocq. In this section, we detail an architecture that supports both student and expert users, allows for a range of automation, is customizable and extensible, and permits export to a theorem prover.

3.1 Rewrite Search

Our interface has three automation modes: One Step, Check Result, and Suggest Result, which we detail below. One Step (Figure 1) is the most restricted mode, in which users must take a single step at a time: for example, distributing $(x + 1)(x + 1)$ to $x^2 + x + x + 1$. The nature of what constitutes a “step” is defined by the current math profile (see Section 3.2). For example, this distribution could be more verbose: $(x + 1) \cdot x + (x + 1) \cdot 1$, or less verbose: $x^2 + 2x + 1$. One Step does not merely require that the proposed result be mathematically correct: it must be reachable by one enabled rewrite. Thus, evaluating $3 + 4 \cdot 4$ directly to 19 is rejected because it combines multiplication and addition, whereas evaluating the selected subexpression $4 \cdot 4$ to 16 is accepted.

The Check Result and Suggest Result modes (Figure 2) are aimed more at expert users, with Suggest Result extending Check Result to automatically simplify the selected subexpression. In Check Result mode, the user proposes an equivalent result that may require multiple rewrites, and the interface checks its validity. Validity depends on the current math profile. For example, a user may select $(2x - 3)(x + 4)$ and enter $2x^2 + 5x - 12$. Check Result may accept this as one interaction, but only if the active profile permits the distribution and cleanup needed to reach that result. If collecting like terms is disabled, mathematical equivalence alone does not make the result acceptable.

3.2 Math Profiles and Automation Policy

Exercises are associated with a *math profile*, which defines the enabled rewrites and automation. Those rewrites are suggested to the user contextually: after a user selects a subexpression, Hazel offers identities and rules whose patterns match that selection. For example, a rule for $\sin^2(\cdot)$ is offered only when the selected term has that form. Users choose among applicable suggestions. The same profile bounds Check

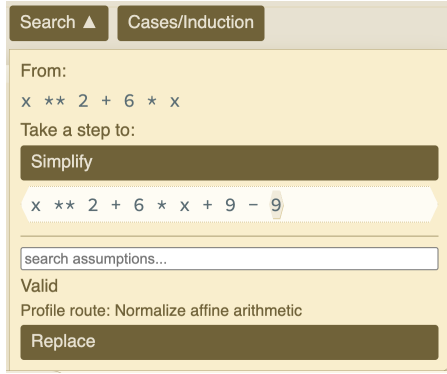


Figure 2. In Check Result and Suggest Result modes, the interface checks the validity of a suggestion produced by a validated sequence of rewrites.

Result search: Hazel accepts a result only if a sequence of permitted rewrites reaches it.

Our design is oriented around the principle that a student who has moved on to a later mathematical level has mastered and can abstract away more primitive skills. For example, a student learning trigonometry need not verbosely compute arithmetic according to the order of operations or manually distribute binomials. As such, we envision progressing through math levels iteratively: first increasing automation (algebraic simplification, factoring, etc.) as skills are learned, and then “unlocking” new mathematical capabilities when introducing a new subject such as trigonometry.

3.3 Proof Export

A completed sequence of rewrite steps in the Hazel interface constitutes a structured but informal proof. We export these step sequences to the Rocq theorem prover for machine-checked verification [15]. An exported proof uses the rewrite rules from each step, either taken directly in One Step mode or found by search in more automated modes. The Rocq proof assistant is integrated in two ways: first, in the browser, where JSCoq (a web interface for the Rocq theorem prover) is used to check the validity of each step, and second, through an export feature that generates a complete proof script corresponding to all steps taken [3]. In Check Result and Suggest Result modes, after a rewrite sequence has been found using search, the interface generates a Rocq theorem and sends it to a background JSCoq worker to check.

4 Extended Case Studies

This section presents several case studies of increasing complexity and evaluates our interface against them. The case studies are inspired by courses spanning algebra education through calculus, and are ordered so that each builds on capabilities established by the prior one. The simplest case studies are intended to be valuable teaching tools where proof export is trivial using automated tactics; our later case studies

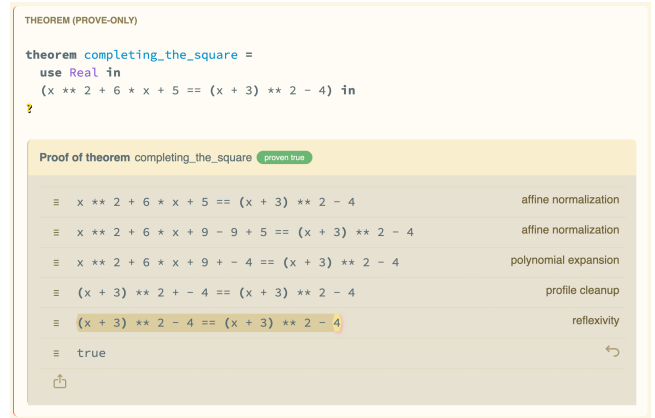


Figure 3. Completing the square in Hazel Prover.

aim to simplify formal proof for numerical methods practitioners. Due to space constraints, only select case studies are described below. In each case study, we briefly comment on the experience of using the interface and the extent to which it meets our design criteria (Section 2). Our design criteria about *low setup cost* are consistently met across case studies: the exercises are browser-based and interface elements to take rewrites are presented upon selecting terms.

Order of Operations. The first case study asks students to simplify the arithmetic expression $3 + 4 \cdot 2^2 - \frac{6}{3}$ step by step to obtain 17, while obeying the order of operations. It uses One Step, the most verbose mode.

The verbose steps in this case study are by design, given the target student audience. In this way, our goals around *low interaction overhead* and *step granularity* are met: users must take small steps, each corresponding to one arithmetic operation, in a sequence that obeys the order of operations. The *mathematical coverage* goal is trivially met by built-in arithmetic operations.

Completing the Square. Targeted at algebra courses (Figure 3), users rewrite $x^2 + 6x + 5$ as $(x+3)^2 - 4$ by completing the square. Users select $x^2 + 6x$ and rewrite it as $x^2 + 6x + 9 - 9$, simplify $-9 + 5$ to -4 , and factor $x^2 + 6x + 9$ as $(x+3)^2$. Cleanup and reflexivity close the proof.

This case study is an example of where an expression may need to become more complex (adding $+9 - 9$, here) in order to complete the square. In Check Result mode, this case study permits that exact operation to be taken, and a second step to factor the square $x^2 + 6x + 9$ as $(x+3)^2$. The steps match work on paper, meeting our goals of *interaction overhead*, *step granularity*, and *mathematical coverage*.

Trigonometric Identities. This case study targets rewriting trigonometric expressions, specifically showing

$$1 + 2 \sin^4(x) = \frac{7}{4} - \cos(2x) + \frac{1}{4} \cos(4x).$$

Trigonometric identities introduce a challenge absent from algebraic manipulation: at many intermediate steps, multiple valid rewrites apply to the same subexpression. For instance,

$\sin^2(x)$ can be rewritten using the Pythagorean identity or a double-angle formula; our interface presents both. This case study targets both precalculus education and numerical methods practitioners who routinely manipulate trigonometric expressions in derivations but lack tooling that records and verifies each step.

Automatic suggestion and application of applicable identities from our interface, combined with simplification of the resulting double-angle expression, reduces the tedium of finding and simplifying trigonometric identities. In this case, our interface reduces *step granularity* in a desirable way compared to pen-and-paper work, with automatic suggestions keeping *interaction overhead* low.

Quadratic Taylor Approximation. This case study constructs the (exact) second-order Taylor polynomial of $f(t) = t^2 + 3t + 2$ about $t = 0$. A series of dependent `let` bindings defines the function and its first two derivatives: `f1` differentiates `f`, and `f2` in turn differentiates `f1`. Users first differentiate f and bind the result $f_1(t) = 2t + 3$. The interface then substitutes this computed function for f_1 in the next derivative expression and differentiates it again, producing $f_2(t) = 2$. Users reuse these results, evaluate the derivatives at a given point, and assemble the finite Taylor polynomial. This case study targets college-level Calculus II.

In this case study and the next, we encounter limitations of the interface. At the moment, only whole “programs” can be stepped, not individual subexpressions. Therefore, steps such as computing the first derivative `f1` require stepping a larger program that computes `f2` and the Taylor approximation. In this specific polynomial example, the step burden remains small, as further derivatives reduce in degree towards constants. However, for more complex functions, the step burden can be significant, like in the following case study. As such, *interaction overhead* becomes a concern, though *step granularity* still permits computing derivatives of interest at each step. In this specific case, *mathematical coverage* is as desired, but the following case study lays out shortcomings.

Trigonometric Taylor Approximation. The final case study asks users to compute a Taylor polynomial for the trigonometric function from the trigonometry case study, $\frac{7}{4} - \cos(2x) + \frac{1}{4}\cos(4x)$. This more complex function is targeted at real-world numerical methods practitioners. Though similar to the previous case study, the work is much more tedious due to the complexity of the function and number of chain rule derivatives, leading to high *interaction overhead*. This represents a failure mode of our interface at present; support for stepping and substituting subexpressions would reduce the tedium. Rocq proof export does not yet support Taylor remainders, preventing formal proof export and limiting *mathematical coverage*.

References

- [1] Michael J. Beeson. 1998. Design Principles of Mathpert: Software to Support Education in Algebra and Calculus. In *Computer-Human*

- Interaction in Symbolic Computation*. Springer Vienna, 163–177.
- [2] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE 28*. Springer-Verlag, Berlin, Heidelberg, 625–635.
- [3] Emilio Jesús Gallego Arias, Benoît Pin, and Pierre Jouvelot. 2017. jsCoq: Towards Hybrid Theorem Proving Interfaces. *Electronic Proceedings in Theoretical Computer Science* 239 (2017), 15–27.
- [4] Markus Hohenwarter and Judith Preiner. 2007. Dynamic Mathematics with GeoGebra. *Journal of Online Mathematics and its Applications* 7 (2007). Article 1448.
- [5] Matthew Keenan, Nishant Kheterpal, Jean-Baptiste Jeannin, and Cyrus Omar. 2026. Hazel Prover: A Classroom Proof Assistant for Learning Structural Induction. arXiv:2608.23309 [cs.PL]
- [6] Maplesoft, a division of Waterloo Maple Inc. 2026. Maple 2026. Waterloo, Ontario, Canada. <https://www.maplesoft.com/products/maple/>
- [7] Patrick Massot. 2024. Teaching Mathematics Using Lean and Controlled Natural Language. In *ITP (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 27:1–27:19.
- [8] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. 2017. SymPy: symbolic computing in Python. *PeerJ Computer Science* 3 (2017), e103.
- [9] Frédéric Tran Minh, Laure Gonnord, and Julien Narboux. 2025. A Lean-based Language for Teaching Proof in High School. In *CICM*. Springer, 447–467.
- [10] Tobias Nipkow, Markus Wenzel, and Lawrence C Paulson. 2002. *Isabelle/HOL: a proof assistant for higher-order logic*. Springer.
- [11] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. 2019. Live functional programming with typed holes. *Proc. ACM Program. Lang.* 3, 3, POPL (2019), 14:1–14:32.
- [12] Erin R. Ottmar, David Landy, Erik Weitnauer, and Robert L. Goldstone. 2015. Graspable Mathematics: Using Perceptual Learning Technology to Discover Algebraic Notation. In *Integrating Touch-Enabled and Mobile Devices into Contemporary Mathematics Education*. IGI Global, Hershey, PA, 24–48.
- [13] Benoit Rognier and Guillaume Duhamel. 2016. Présentation de la plateforme edukera. In *JFLA*. <https://hal.science/hal-01333606>
- [14] The MathWorks, Inc. 2025. *Symbolic Math Toolbox*. Natick, Massachusetts, USA. <https://www.mathworks.com/products/symbolic.html>
- [15] The Rocq Development Team. 2025. *The Rocq Prover, Version 9.0.0*. Formerly known as the Coq Proof Assistant. <https://rocq-prover.org/releases/9.0.0>
- [16] Aalt Jelle Wemmenhove. 2025. Waterproof: Transforming a proof assistant into an educational tool. (2025).
- [17] Jelle Wemmenhove, Dick Arends, Thijs Beurskens, Maitreyee Bhaid, Sean McCarren, Jan Moraal, Diego Rivera Garrido, David Tuin, Malcolm Vassallo, Pieter Wils, et al. 2022. Waterproof: educational software for learning how to write mathematical proofs. *arXiv preprint arXiv:2211.13513* (2022).
- [18] Michael L. Winer and Michael T. Battista. 2022. Investigating Students’ Proof Reasoning: Analyzing Students’ Oral Proof Explanations and Their Written Proofs in High School Geometry. *International Electronic Journal of Mathematics Education* 17, 2 (2022), em0677.
- [19] Wolfram Research Inc. 2026. Mathematica, Version 15.0. Champaign, IL. <https://www.wolfram.com/mathematica>